ClickSense

Maintaining ClickSense, step by step

Maintenance guide — version 1.0.0

This guide is for the people who keep ClickSense useful once it is installed: change a setting, document a new table, add a metric, a business term, a verified query, a rule or a playbook, and review what the agent suggests. Every recipe gives the exact commands, what to check, and what "done" looks like.

The examples use the demo database (`shop`). Commands are written for macOS/Linux (`clicksense-admin ...`, run from the project folder with the virtual environment active, or `.venv/bin/clicksense-admin ...`). On the Windows server, use `admin-windows.bat ...` with the same arguments, from `C:\clicksense`.

Contents

0. Before you start 3

- Where things live . . . 3
- The three commands you will use every time . . . 3
- Golden rules . . . 3

1. Change a setting 3

2. Document a new table 4

3. Teach the agent your users' words for stored values 5

4. Add a governed metric 5

5. Add a business term (and handle ambiguous words) 6

6. Add a verified example query 6

7. Add a business rule (knowledge document) 7

8. Add or change a playbook (skill) 7

9. Review what the agent suggests 8

10. Ship a change 8

11. Troubleshooting 8

12. Import a whole existing knowledge base 9

Appendix: what each command does 9

0. Before you start

Where things live

You want to change	Edit	Takes effect
A setting (limits, connection, HTTP, LLM)	`clicksense.yaml`	after a **restart**
A secret (password, API key, HTTP token)	`.env`	after a **restart**
Tables, columns, values, synonyms	`semantic/dictionary/<db>.yaml`	**within seconds**, no restart
Metrics	`semantic/metrics.yaml`	within seconds
Business terms	`semantic/glossary.yaml`	within seconds
Verified queries	`semantic/examples.yaml`	within seconds
Business rules	`semantic/knowledge/<name>.md`	within seconds
Playbooks	`semantic/skills/<name>/SKILL.md`	within seconds

How to restart: stop and start `clicksense` (or `serve-windows.bat`); for the Windows service, from an elevated PowerShell:

```
powershell -ExecutionPolicy Bypass -File scripts\windows\service.ps1 restart
```

The three commands you will use every time

```
clicksense-admin kb validate --live        # is everything well formed and still true in ClickHouse?
clicksense-admin kb search "a question"    # what will the agent find for this question?
clicksense-admin kb resolve TABLE COLUMN word1 word2   # how do users' words map to stored values?
```

`kb validate` exits with status 1 when there is an error, so it can run in CI. Without `--live` it only checks the files (useful when ClickHouse is not reachable).

Golden rules

1. **Write for a new analyst.** The agent reads your text as-is: one clear sentence beats a paragraph of jargon.
2. **Never leave a `TODO`.** Templates are full of them on purpose; `kb validate` flags any left.
3. **Validate after every change**, then **search** with a real question.
4. **Version everything.** `semantic/` is plain text: commit it, review it in pull requests.

1. Change a setting

Every setting is documented in one place, generated from the code:

```
clicksense-admin config --example     # all keys, defaults and meanings (= clicksense.example.yaml)
clicksense-admin config --show        # what the server actually uses now (secrets masked)
```

Steps

1. Find the key in `config --example` (e.g. `limits.max_execution_time`, "Seconds before ClickHouse aborts a query").
2. Put it in `clicksense.yaml`, under its section. Keep only the keys you change:

```
limits:
  max_execution_time: 40
  preflight_max_rows: 800000000
```

3. If the key has an environment variable (shown as `[env: ...]`), you may set it in `.env` instead; the environment always wins over the YAML. Secrets **must** go in `.env`.
4. Run `clicksense-admin config --show` and check the value.
5. Restart the server.

Common changes

Symptom	Setting
Legitimate queries refused as `too_expensive`	`limits.preflight_max_rows` (raise)
Heavy analyses time out	`limits.max_execution_time` (raise, within the ClickHouse profile's MAX)
Answers cut at 200 rows	`limits.default_rows` / the `max_rows` argument
`busy` or `rate_limited` during bursts	`limits.max_concurrent`, `limits.rate_per_minute` (only if the cluster has headroom)
The agent should see only some databases	`policy.allowed_databases`
Hide a table	`policy.denied_tables: ["db.table"]`
Other machines must connect	`.env`: `CLICKSENSE_HTTP_HOST=0.0.0.0` and a `CLICKSENSE_HTTP_TOKENS` value

> **Limits can only tighten the ClickHouse account.**
> If you raise a limit above the profile's `MAX` (see `dev/production-account.sql`), every query fails with `setting_constraint`. Raise the profile first, then the setting.

Done when `config --show` shows the new value and `clicksense --check` is happy.

2. Document a new table

Steps

1. Let ClickHouse write the skeleton:

```
clicksense-admin kb bootstrap shop.returns
```

This appends `shop.returns` to `semantic/dictionary/shop.yaml` with: the table COMMENT, the time column guessed from the keys, every column with its COMMENT, likely PII columns (guessed from their names, marked `# TODO confirm`), and the stored values of low-cardinality text columns (with the value itself as a placeholder label). Use `--print-only` to see it first.

2. Open the file and fill in, in this order:
   - `description` and `grain` ("one row per returned item");
   - `caveats`: the traps ("amount is negative for refunds");
   - for each column users filter on: replace the placeholder labels in `values` with meanings, and add `value_synonyms` with the words people use (see recipe 3);
   - confirm or remove each guessed `pii: true`;
   - `joins` and `dimensions` if metrics will group by a column of another table.
3. Validate and test:

```
clicksense-admin kb validate --live
clicksense-admin kb search "how many returns last month by reason"
```

Done when validate shows no error or TODO, and the search ranks the table first.

3. Teach the agent your users' words for stored values

The most common wrong answer: the user says "France", the column stores FR; the query runs and returns nothing. Fix it once in the dictionary.

Steps

1. See how words resolve today:

```
clicksense-admin kb resolve shop.orders country France germany Narnia
```

```
'France'                resolved   -> 'FR'  'FR' (label, 0.96)
'germany'               resolved   -> 'DE'  'DE' (label, 0.96)
'Narnia'                not_found
```

2. For a word that is `not_found` or `ambiguous` but should map to a value, edit the column:

```
country:
  values:                # stored value: what it means
    FR: France
    GB: United Kingdom
  value_synonyms:        # stored value: [words users say]
    GB: [uk, britain, england]
```

Labels and synonyms are matched case-, accent- and punctuation-insensitively; synonyms can be in any language.

3. Run `kb resolve` again: the word should now be `resolved`.
4. If a value list is long and you only documented part of it, add `values_complete: false`, so the lint does not report the other stored values as wrong.

Done when every word users commonly say resolves to the right value.

4. Add a governed metric

A metric fixes *one* definition of a number ("revenue") so every answer uses it.

Steps

1. Scaffold it:

```
clicksense-admin kb new metric net_revenue
```

2. Fill the new entry at the end of `semantic/metrics.yaml`:

```
- name: net_revenue
  label: Net revenue
  description: Revenue excluding VAT of delivered order lines (amount / 1.2).
  table: shop.orders
  expression: round(sum(o.amount_eur) / 1.2, 2)   # alias `o` comes from the dictionary
  filters: ["o.status = 'completed'"]
  time_column: order_date
  unit: EUR
  synonyms: [revenue excluding vat, net sales]
  dimensions: [country, channel, category, segment]
```

- `expression` is an aggregate over the table, using the table's `alias`.
- `dimensions` may be columns of the table or `dimensions:` declared in the dictionary (`category` is `p.category` through the `products` join).
- A word must point to **one** metric. If users say "sales" for several metrics, do not put it in `synonyms`: make it an ambiguous glossary term (recipe 5).

3. Validate — `--live` compiles the metric and has ClickHouse check the SQL:

```
clicksense-admin kb validate --live
```

4. Try it through the agent (`clicksense-chat -q "net revenue by channel last month"`) or check that `kb search "net revenue"` lists it.

Done when validate is clean and the figure matches a hand-written query.

5. Add a business term (and handle ambiguous words)

Unambiguous term — a word with one meaning:

```
clicksense-admin kb new term "basket size"
```

```
- term: basket size
  synonyms: [average basket]
  definition: Revenue divided by the number of delivered order lines.
  maps_to: "metric:average_order_value"
```

Ambiguous term — one word, several meanings. List them under `senses` and remove `maps_to`. When a question contains the word, `get_context` reports it and the agent asks the user:

```
- term: sales
  definition: Ambiguous word — it can mean money, orders or units.
  senses:
    - {meaning: Revenue in EUR, maps_to: "metric:revenue"}
    - {meaning: Number of orders, maps_to: "metric:orders"}
    - {meaning: Units sold, maps_to: "metric:units_sold"}
```

Then `kb validate` (it checks that every `metric:` target exists) and `kb search "sales in France"`: the term must appear under "Ambiguities".

6. Add a verified example query

Examples are the closest thing to teaching by demonstration: the agent receives the most similar ones with each question. Only add SQL you would sign.

```
clicksense-admin kb new example "Revenue by brand last week"
```

```
- question: "Revenue by brand last week"
  tags: [revenue, brand, weekly]
  sql: |
    SELECT p.brand, sum(o.amount_eur) AS revenue
    FROM shop.orders AS o
    INNER JOIN shop.products AS p ON o.product_id = p.product_id
    WHERE o.status = 'completed'
      AND o.order_date >= toMonday(today()) - 7 AND o.order_date < toMonday(today())
    GROUP BY p.brand ORDER BY revenue DESC
  notes: Weeks start on Monday.
```

`kb validate --live` re-runs the guard and has ClickHouse check every example, so an example broken by a schema change is caught. Run it after any schema migration.

7. Add a business rule (knowledge document)

```
clicksense-admin kb new knowledge refund-policy
```

This creates `semantic/knowledge/refund-policy.md`:

```
---
id: refund-policy
title: How refunds are counted
tags: [refund, revenue, finance]
applies_to: [shop.orders]
---
Refunded lines keep status = 'refunded' and are excluded from revenue. Finance reports refunds
separately, by refund date, which is not in ClickHouse yet.

To count refunds in a period, filter status = 'refunded' on order_date and say in the answer
that the date is the order date, not the refund date.
```

Write short paragraphs that stand on their own: the agent receives the paragraph that best matches the question. `applies_to` makes `describe_table` list the document.

8. Add or change a playbook (skill)

Skills are step-by-step procedures for recurring analyses, in the standard Agent Skills format. They are offered to clients as tools (`get_skill`), as prompts (`skill-<name>`) and as native MCP skills.

```
clicksense-admin kb new skill weekly-review
```

```
---
name: weekly-review
description: Weekly sales review — revenue, orders and basket vs last week, with the main drivers.
when_to_use: weekly review, week over week, monday report, revue hebdo
---
1. Call `query_metric` for revenue, orders and average_order_value with period last_week, then
   with the week before (period {"from": ..., "to": ...}).
2. Compute the change of each metric in % and in absolute value.
3. For revenue, break down by channel and by category; keep the 3 largest contributions.
4. Answer: a 3-line summary, a table, then the drivers. State the periods and definitions.
```

- `description` decides when a client loads the skill: make it specific.
- `when_to_use` is searched: put the words your users actually say (several languages are fine).
- Keep the folder name and `name` identical (validate warns otherwise).

9. Review what the agent suggests

When the agent finds something reusable it calls `suggest_knowledge`, which writes a file to `semantic/inbox/`. Nothing there is used until a person moves it.

```
clicksense-admin kb inbox
```

For each suggestion: if it is right, copy it into the right file (use `kb new ...` to get the structure), run `kb validate --live`, then delete the inbox file. If it is wrong, just delete it — and consider adding a caveat or a knowledge rule so the agent does not draw the same conclusion again.

10. Ship a change

1. `clicksense-admin kb validate --live` — no error, no warning.
2. `clicksense-admin kb search "..."` for two or three real questions touched by the change.
3. If you changed code or settings: `python -m pytest -q` (needs the demo database for the live tests, see the README).
4. Commit `semantic/` (and `clicksense.yaml` changes if they are shared), open a pull request.
5. On the server: `git pull`. Semantic files are picked up within seconds; restart only if `clicksense.yaml`, `.env` or code changed.

11. Troubleshooting

Symptom	Cause and fix
`kb validate`: `unknown key 'descripton' is ignored`	Typo: the loader ignores unknown keys, so that text never reached the agent. Fix the key.
`kb validate`: `invalid YAML`	Indentation or an unquoted `:` / `,` in a value. Quote the value: `description: "Order timestamp, Paris time"`.
`kb validate --live`: `the compiled SQL fails`	The metric's `expression`/`filters` use a column or alias that does not exist. Check the table alias in the dictionary.
`kb validate --live`: `the SQL no longer runs`	An example broke after a schema change. Update it: agents copy examples.
The agent picks the wrong table	`kb search` the question; add `synonyms`/`description` to the right table, or a metric/term pointing to it.
The agent filters on a value that does not exist	`kb resolve` the word; add a label or a `value_synonyms` entry (recipe 3).
A change is not visible	Semantic files: wait a few seconds or call `refresh_catalog`. Settings and `.env`: restart.
`server_status` lists knowledge errors	A file failed to load; `kb validate` names it and the line.

12. Import a whole existing knowledge base

If your knowledge already lives in Markdown files (wiki export, data catalogue, team notes), an AI coding agent can apply all the recipes above in one pass. Give it the prompt in PROMPT-import-knowledge.md with the path of the folder: it inventories every file, maps each piece of information to its place in `semantic/`, checks it against ClickHouse, writes the entries from the templates, runs `kb validate --live` and test searches, and opens a pull request listing what it could not integrate and the conflicts to arbitrate. Review that pull request like any other change (recipe 10).

Appendix: what each command does

Command	Purpose
`clicksense-admin config --example`	Commented configuration with every key and default
`clicksense-admin config --show`	Effective configuration (secrets masked)
`clicksense-admin kb validate [--live]`	Lint the semantic layer (`--live`: against ClickHouse)
`clicksense-admin kb new KIND NAME`	Add a table/metric/term/example/knowledge/skill from a template
`clicksense-admin kb bootstrap DB.TABLE [--print-only]`	Document a table from ClickHouse
`clicksense-admin kb search "QUESTION" [--json]`	What the agent finds for a question
`clicksense-admin kb resolve TABLE COLUMN WORD...`	How words map to stored values
`clicksense-admin kb inbox`	The agent's pending suggestions
`clicksense --check`	Connection and server status
`clicksense --gen-token`	A new bearer token for HTTP clients